

Statistical and Machine Learning SS26

Exercise Sheet 3

Exercise 1: Confusion Matrix

```
# (a) Confusion matrix values
TP <- 80
FN <- 20
FP <- 10
TN <- 390

# Accuracy
accuracy <- (TP + TN) / (TP + TN + FP + FN)

# Sensitivity / TPR / Recall
sensitivity <- TP / (TP + FN)

# Specificity
specificity <- TN / (FP + TN)

# Precision
precision <- TP / (TP + FP)

# F1-score (harmonic mean of precision and recall)
f1 <- 2 * (precision * sensitivity) / (precision + sensitivity)

#round, name and print metrics
cat("Accuracy: ", round(accuracy, 4), "\n")

## Accuracy:    0.94

cat("Sensitivity:", round(sensitivity, 4), "\n")

## Sensitivity: 0.8

cat("Specificity:", round(specificity, 4), "\n")

## Specificity: 0.975

cat("Precision: ", round(precision, 4), "\n")

## Precision:   0.8889
```

```
cat("F1-Score:   ", round(f1, 4), "\n")
```

```
## F1-Score:    0.8421
```

Answer (b): In fraud detection, a **false negative** (a real fraud case labelled as legitimate) is very costly — money is lost and the customer is harmed. **Sensitivity (TPR/Recall)** is therefore the most critical metric: we want to catch as many true fraud cases as possible. The F1-score is also informative because it balances precision and recall when the classes are imbalanced.

Exercise 2: ROC Curve and AUC

```
set.seed(42)

# (a) Simulate scores and labels
scores_pos <- rbeta(100, 5, 2) # beta-distribution; values between 0 and 1;
# (alpha > beta -> right-skewed distribution, more values are closer to 1)
scores_neg <- rbeta(100, 2, 5) # (alpha < beta -> left-skewed distribution,
# more values are closer to 0)

scores <- c(scores_pos, scores_neg) # combines both scores to one vector
# -> vector of 200 numbers
labels <- c(rep(1, 100), rep(0, 100)) # creates vector with 100 1s and 100 0s,
# because the first 100 observations are 'true positive' and the second 100 obs.
# are 'true negative'

# (b) Compute TPR and FPR over thresholds
thresholds <- seq(1, 0, by = -0.01) # creates a sequence from 1 down to 0 in steps
# of 0.01: cutoff points the loop will test

#following two commands create empty vector 'containers' for tpr and fpr
# to be filled later in the loop for each threshold.
# it has the same length as the thresholds sequence
tpr <- numeric(length(thresholds)) # true positive rate/sensitivity/recall
fpr <- numeric(length(thresholds)) # false positive rate

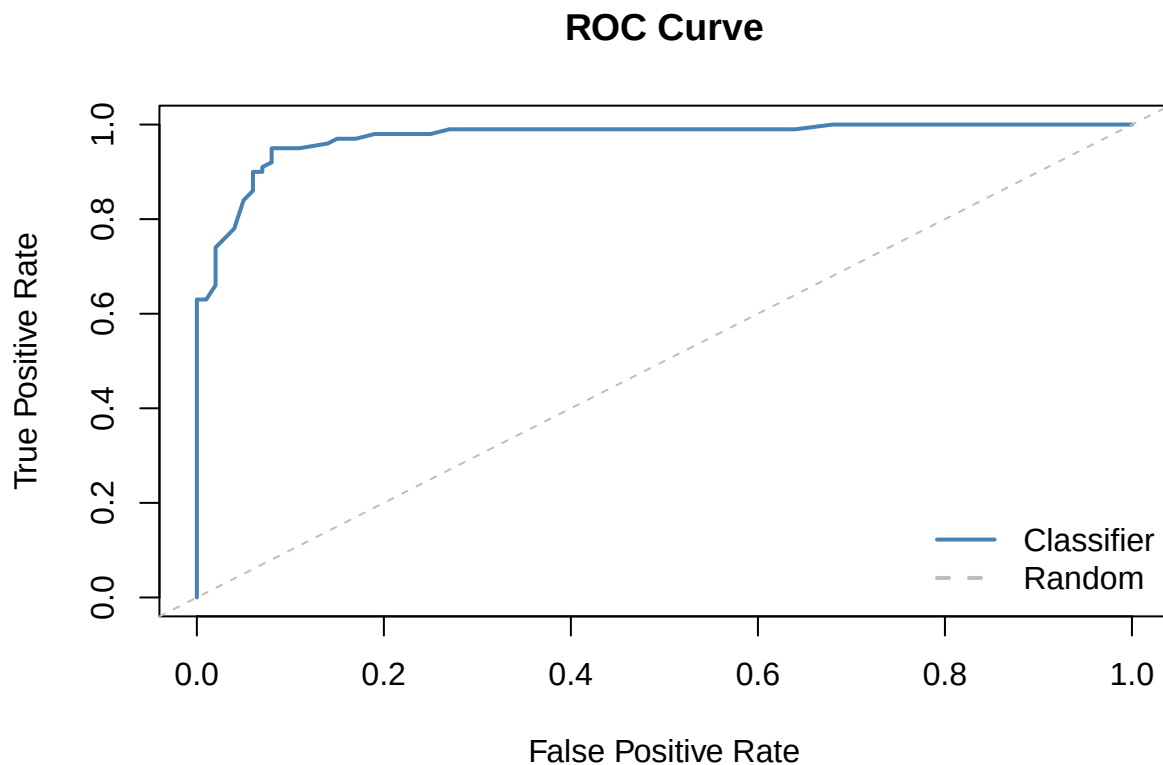
for (i in seq_along(thresholds)) { # loop runs for each threshold value
  t <- thresholds[i] #picks current threshold (eg first 1, then 0.99, then 0.98, ...)
  pred <- ifelse(scores >= t, 1, 0) # For every observation in the `score` vector,
  # a predicted label is assigned.
  # -> `pred` is a new vector of 200 zeros and ones, different for each threshold.
  TP <- sum(pred == 1 & labels == 1) #set up of confusion matrix:
  # comparison of `pred` and `labels` position by position .
  # If true = 1 and pred = 1 -> counts as true positive.
  FP <- sum(pred == 1 & labels == 0) #pred = 1, true = 0 -> false positive
  TN <- sum(pred == 0 & labels == 0) #etc
  FN <- sum(pred == 0 & labels == 1) #etc
  tpr[i] <- TP / (TP + FN) #TPR and FPR are calculated for each threshold and
  # stored in vectors we created before
```

```

fpr[i] <- FP / (FP + TN)
}

# plot FPR on x-axis, TPR on y-axis.
# The dashed diagonal is the random classifier baseline
# x,y,plot as line, line width=2, color steelblue, axis labels, title
plot(fpr, tpr, type = "l", lwd = 2, col = "steelblue",
     xlab = "False Positive Rate", ylab = "True Positive Rate",
     main = "ROC Curve")
# draw diagonal reference line, intercept 0, slope 1, dashed, gray
abline(0, 1, lty = 2, col = "gray")
# add legend: #color; line/dashed; line width, no box
legend("bottomright", legend = c("Classifier", "Random"),
      col = c("steelblue", "gray"), lty = c(1, 2), lwd = 2, bty = "n")

```



```

# (c) AUC using the trapezoidal rule
auc <- sum(diff(fpr) * (head(tpr, -1) + tail(tpr, -1)) / 2) #sum of all t.areas
# (calc: width of each trapezoid * average height of trapezoid)
auc <- abs(auc) # diff(fpr) can be negative (decreasing order), take absolute value
cat("AUC:", round(auc, 4), "\n") # print result

```

```
## AUC: 0.973
```

```
#AUC: area under the ROC curve. measures how well model ranks positives above
# negatives. 1 -> perfect classifier; 0 -> random guessing
# For example, AUC = 0.87 means: If you pick one person who is truly positive
# and one who is truly negative, the model gives the positive a higher score 87% of the time.
# measuring AUC: each small piece between two points on the curve is a trapezoid
# (left/right parts are parallel). Area of one trapezoid = width × (h1 + h2) / 2
```

Answer (c): AUC: 0.973 excellent result! interpretation: The AUC represents the probability that the classifier ranks a randomly chosen positive higher than a randomly chosen negative. —

Exercise 3: Bootstrap Standard Error

```
set.seed(101)

n <- 50 # number of observations we want to draw
x <- rexp(n, rate = 2) # draws n obs. from an exponential distribution with rate = 2

# (b) Bootstrap SE of the median
B <- 1000 # number of resamples
boot_medians <- numeric(B) #pre-allocates 1000 empty slots - one for each resample.

for (b in 1:B) { # for each iteration of the 1000
  x_boot <- sample(x, size = n, replace = TRUE) # draws 50 values with replacement
# from the original sample x (some values appear twice, some not at all) and
# computes the median of that resample
  boot_medians[b] <- median(x_boot) # stores median values in boot_medians[b]
}

boot_se_median <- sd(boot_medians) # standard error of these 1000 medians
cat("Bootstrap SE of median:", round(boot_se_median, 5), "\n")
```

```
## Bootstrap SE of median: 0.04221
```

```
# (c) Analytical SE of mean (for comparison)
sigma_hat <- sd(x)
se_mean <- sigma_hat / sqrt(n) #analytical formula for the SE of the mean
cat("Analytical SE of mean: ", round(se_mean, 5), "\n")
```

```
## Analytical SE of mean: 0.05925
```

```
cat("Sample median: ", round(median(x), 5), "\n")
```

```
## Sample median: 0.27375
```

```
cat("True median Exp(2): ", round(log(2) / 2, 5), "\n")
```

```
## True median Exp(2): 0.34657
```

Answer (c): The analytical SE for the **mean** has a closed-form formula ($\sigma/\sqrt{n}$). The **median** does not have such a simple formula for arbitrary distributions. The bootstrap is particularly valuable here because it estimates the sampling variability of *any* statistic — including the median — purely from the data, without requiring distributional assumptions or closed-form derivatives.

Exercise 4: K-Fold Cross-Validation from Scratch

```
set.seed(99)
#a)
n <- 200
x <- runif(n, 0, 5) # creates 200 random x values between 0 and 5
y <- 1 + 2 * x - 0.5 * x^2 + rnorm(n, sd = 1) # simulation of data y incl. random noise

dat <- data.frame(x = x, y = y) # creates data frame with x and corresponding y value

#b)
# 5-fold CV helper
cv_mse <- function(formula, data, K = 5) {
  folds <- sample(rep(1:K, length.out = nrow(data))) # randomly assigns each of the 200
  # observations to one of 5 folds
  mse_vec <- numeric(K) # creates 5 empty containers for 5 MSE values
  for (k in 1:K) { #loop runs 5 times (for each k)
    train <- data[folds != k, ] # all folds except k -> 160 rows
    test <- data[folds == k, ] # only fold k -> 40 rows
    fit <- lm(formula, data = train) # trains model on 160 rows
    pred <- predict(fit, newdata = test) # uses the trained model to predict on the 40 test rows
    mse_vec[k] <- mean((test$y - pred)^2) # computes MSE for each k:
    # actual y value - predicted y value of model, squared, mean of this -> MSE
  }
  mean(mse_vec) # averages 5 MSE values
}

# Every observation is used exactly once as test data!
# that's what makes cross-validation a reliable estimate of real-world model performance

mse_linear <- cv_mse(y ~ x, dat)
mse_quadratic <- cv_mse(y ~ x + I(x^2), dat)

cat("CV-MSE (linear): ", round(mse_linear, 4), "\n")

## CV-MSE (linear): 1.7711

cat("CV-MSE (quadratic): ", round(mse_quadratic, 4), "\n")

## CV-MSE (quadratic): 1.1685
```

Answer (c): The **quadratic model** should yield a substantially lower CV-MSE because the true data-generating process is quadratic. The linear model underfits (high bias), whereas the quadratic model captures the true curvature.